

TYMScannerLib

使 用 手 册

作 者：_____

完成日期：_____

签 收 人：_____

签收日期：_____

修改情况记录：

版本号	修改批准人	修改人	修改日期	签收人
V5.1.0		GAOJUN	20240122	
V5.2.0		GAOJUN	20240325	
V5.3.0		GAOJUN	20240424	
V5.4.0		GAOJUN	20240613	
V5.5.0		GAOJUN	20240620	
V5.6.0		GAOJUN	20240723	
V5.7.0		GAOJUN	20240814	
V5.8.0		GAOJUN	20241118	
V5.9.0		GAOJUN	20241210	

目录

使 用 手 册	1
1 引言	3
1.1 编写目的.....	3
1.2 定义.....	3
2 用途	3
3 函数速览.....	3
4 函数说明.....	5
5 结构体说明.....	32
6 返回值说明.....	39
7 校准流程说明.....	40
8 特殊说明.....	41
9 实时状态信息获取.....	42
10 系统快速使用流程图.....	46
11 示例代码.....	47
12 数据格式.....	49

1 引言

1.1 编写目的

为了使数据采集板的软件开发人员，理解和使用动态库 TYMScannerLib.dll，我们编写了本使用手册。

1.2 定义

列出本文件中用到的专门术语的定义和缩写词的原词组。

2 用途

TYMScannerLib 的主要功能是通过 UDP 协议控制数据采集板，接收数据采集板发出的图像数据包。并对接收到的数据做解包处理。在此基础上向应用层软件（调用方）提供一个清晰的，运行稳定的数据采集板的开发库 API。

3 函数速览

函数名	是否必需	备注
tymscan_version	否	
tymscan_init	是	
tymscan_inited	否	
tymscan_cleanup	是	
tymscan_version_firmware	否	
tymscan_connected	是	
tymscan_grab_start	是	
tymscan_grab_stop	是	
tymscan_get_gain_range	否	
tymscan_set_gain_low	否	用户根据实际应用进行设置
tymscan_get_gain_low	否	用户根据实际应用进行设置
tymscan_set_gain_high	否	用户根据实际应用进行设置

tymscan_get_gain_high	否	用户根据实际应用进行设置
tymscan_get_test_pattern_range	否	
tymscan_set_test_pattern	否	
tymscan_get_test_pattern	否	
tymscan_get_integral_time_range	否	
tymscan_set_integral_time	否	用户根据实际应用进行设置
tymscan_get_integral_time	否	用户根据实际应用进行设置
tymscan_set_daq_ip	否	
tymscan_get_daq_ip	否	
tymscan_set_daq_port	否	
tymscan_get_daq_port	否	
tymscan_set_pixelnum_percard	否	
tymscan_get_pixelnum_percard	否	
tymscan_get_pixelnum	否	用户根据实际应用进行设置
tymscan_set_trigger_mode	否	用户根据实际应用进行设置
tymscan_get_trigger_mode	否	用户根据实际应用进行设置
tymscan_set_energy_mode	否	
tymscan_get_energy_mode	否	
tymscan_set_cardnumber	是（在有限情况下）	针对 PANGU-SE 系统，用户必须设置；对于其他系统，用户可根据实际应用选择是否设置
tymscan_get_cardnumber	是（在有限情况下）	针对 PANGU-SE 系统，用户必须设置；对于其他系统，用户可根据实际应用选择是否设置
tymscan_set_datacallback	是	
tymfn_datacallback	是	
tymscan_special_set	否	
tymscan_special_get	否	
tymcan_doOffsetCalibration	否	
tymcan_doGainCalibration	否	
tymcan_setBaseLine	否	
tymcan_set_sendCalibratedData_Enable	否	
tymscan_set_pixel_order_mode	否	
tymscan_save_config	否	
tymscan_set_infopage	否	此为用户定制功能
tymscan_get_infopage	否	此为用户定制功能
tymscan_set_TH_enable_XH	否	此为用户定制功能
tymscan_set_custom_pixnum	否	此为用户定制功能
tymscan_set_detector_type	否	此为 PANGU-GS 系统独有函数

tymscan_get_detector_type	否	此为 PANGU-GS 系统独有函数
tymscan_enable_auto_get_cardnum	否	
tymscan_is_auto_get_cardnum	否	
tymscan_get_total_lost_lines	否	
tymscan_is_attribute_exist	否	
tymscan_get_attribute_value	否	
tymscan_set_work_mode	否	帮助用户快速设定必要参数
tymscan_get_work_mode	否	
tymscan_get_hardware_info_file	否	
tymscan_get_hardware_info_length	否	
tymscan_get_hardware_info	否	
tymscan_set_status_callback	否	
tymscan_is_support_set_gain_per_card	否	
tymscan_get_fcm_firmware_version	否	
tymscan_pdc_set_pitch_and_gaps	否	
tymscan_pdc_enable	否	
tymscan_verify_fcm_info	否	
tymscan_set_fcm_info	否	
tymscan_get_fcm_info	否	
tymscan_enable_interpolation	否	
tymscan_get_detector_type_str	否	
tymscan_get_detector_type_all_fcms_length	否	
tymscan_get_detector_type_all_fcms	否	
tymscan_get_status_info_all	否	
tymscan_set_calib_timeout_for_extrig	否	

4 函数说明

函数：int tymscan_version(char* buf, int buf_length)

说明：获得 sdk 的版本号

参数：

[out] buf: 待存版本号的 buffer

[in] buf_length: buffer 的长度

函数：int tymscan_init(const char* ipaddress_local, const char* ipaddress_daq, int cmd_port, int data_port, int* instance)

说明：初始化数据采集板

参数：

[in] ipaddress_local: 本机端的 ip 地址
[in] ipaddress_daq: 数据采集板的 ip 地址
[in] cmd_port: 指令端口号
[in] data_port: 图像端口号
[out] instance: 返回数据采集板实例 id

函数：int tymscan_initied(bool* initied ,int instance)

说明：询问数据采集板是否初始化成功

参数：

[out] initied: 返回 true 为数据采集板初始化成功，返回 false 为数据采集板初始化失败
[in] instance: 数据采集板实例 id

函数：int tymscan_cleanup(int instance)

说明：清理数据采集板实例

参数：

[in] instance: 数据采集板实例 id

函数：int tymscan_version_firmware(char* version , int buf_length ,int instance)

说明：获取数据采集板固件版本号

参数：

[out] version: 返回数据采集板固件版本号
[in] buf_length: buffer 的长度
[in] instance: 数据采集板实例 id

函数：int tymscan_connected(bool* connected ,int instance)

说明：询问数据采集板是否建立网络连接

参数：

[out] connected: 返回 true 为数据采集板建立连接成功，返回 false 为数据采集板建立连接失败

[in] instance: 数据采集板实例 id

函数：int tymscan_grab_start(int instance,int count=0)

说明：开始采集图像数据

参数：

[in] instance: 数据采集板实例 id

[in] count: 缺省值为 0，此时是连续采集数据。若设置为大于 0 的其他数值，则触发指定次数的回调函数后自动停止采集。

函数: `int tymscan_grab_stop(int instance)`

说明: 停止采集图像数据

参数:

[in] instance: 数据采集板实例 id

函数: `int tymscan_get_gain_range(int* gain_min, int* gain_max, int instance)`

说明: 获取增益范围

参数:

[out] gain_min: 增益最小值

[out] gain_max: 增益最大值

[in] instance: 数据采集板实例 id

函数: `int tymscan_set_gain_low(int card_no, int gain, int instance, int switch_code = 0)`

说明: 设置低能增益

参数:

[in] card_no: 探测器卡的序号。card_no 为 0，表明所有探测器卡设置相同增益。

若想判断该产品是否支持给每块探测器卡设置不同的增益，请调用函数：

`tymscan_is_support_set_gain_per_card`

特殊说明: PANGU-SE 产品虽不支持给每块探测器卡设置不同的增益，但支持通道 1 的所有探测器卡与通道 2 的所有探测器卡有不同的增益配置

例子:

设置通道 1 的所有探测器卡的低能增益:

`tymscan_set_gain_low(1 << 8 | 0, gain, instance_id);` //15bit~8bit 用于告知采集板选择通道 1 来设置增益。

设置通道 2 的所有探测器卡的低能增益:

`tymscan_set_gain_low(2 << 8 | 0, gain, instance_id);` //15bit~8bit 用于告知采集板选择通道 2 来设置增益。

[in] gain: 增益值

[in] instance: 数据采集板实例 id

[in] switch_code: 缺省值为 0。非特殊情况，默认即可。switch_code 有 4 个可选设置值，分别是 0,1,2,3。0 表示: shift off, test off。1 表示: shift off, test on。2 表示: shift on, test off。3 表示: shift on, test on。

函数: `int tymscan_get_gain_low(int card_no, int* gain, int instance, int bufsize=1, int* switch_code = nullptr)`

说明: 获取低能增益

参数:

[in] card_no: 探测器卡的序号。若 card_no 为 0, 表示一次获取所有探测器卡的增益值。
特殊说明: PANGU-SE 产品虽不支持给每块探测器卡设置不同的增益, 但支持通道 1 的所有探测器卡与通道 2 的所有探测器卡有不同的增益配置

例子:

读取通道 1 的探测器卡的低能增益:

tymscan_get_gain_low(1 << 8 | 0, &gain, instance_id); //15bit~8bit 用于告知采集板选择通道 1 来读取增益。

读取通道 2 的探测器卡的低能增益:

tymscan_get_gain_low(2 << 8 | 0, &gain, instance_id); //15bit~8bit 用于告知采集板选择通道 2 来读取增益。

[out] gain: 返回该探测器卡低能增益值

[in] instance: 数据采集板实例 id

[in] bufsize: 设置存储返回的增益值的数组大小。缺省值为 1.

[out] switch_code: 返回目前的 shift 与 test 状态。返回 0, 表示 shift off, test off. 返回 1 表示 shift off, test on. 返回 2 表示 shift on, test off. 返回 3 表示 shift on, test on.

函数: int tymscan_set_gain_high(int card_no, int gain, int instance, int switch_code = 0)

说明: 设置高能增益

参数:

[in] card_no: 探测器卡的序号。card_no 为 0, 表明所有探测器卡设置相同增益。

若想判断该产品是否支持给每块探测器卡设置不同的增益, 请调用函数:

tymscan_is_support_set_gain_per_card

特殊说明: PANGU-SE 产品虽不支持给每块探测器卡设置不同的增益, 但支持通道 1 的所有探测器卡与通道 2 的所有探测器卡有不同的增益配置

例子:

设置通道 1 的所有探测器卡的高能增益:

tymscan_set_gain_high(1 << 8 | 0, gain, instance_id); //15bit~8bit 用于告知采集板选择通道 1 来设置增益。

设置通道 2 的所有探测器卡的高能增益:

tymscan_set_gain_high(2 << 8 | 0, gain, instance_id); //15bit~8bit 用于告知采集板选择通道 2 来设置增益。

[in] gain: 增益值

[in] instance: 数据采集板实例 id

[in] switch_code: 缺省值为 0。非特殊情况, 默认即可。switch_code 有 4 个可选设置值, 分别是 0, 1, 2, 3。0 表示: shift off, test off。1 表示: shift off, test on。2 表示: shift on, test off。3 表示: shift on, test on。

函数: int tymscan_get_gain_high(int card_no, int* gain, int instance, int bufsize=1, int* switch_code = nullptr)

说明: 获取高能增益

参数:

[in] card_no: 探测器卡的序号。若 card_no 为 0, 表示一次获取所有探测器卡的增益值。
特殊说明: PANGU-SE 产品虽不支持给每块探测器卡设置不同的增益, 但支持通道 1 的所有探测器卡与通道 2 的所有探测器卡有不同的增益配置

例子:

读取通道 1 的探测器卡的低能增益:

tymscan_get_gain_high(1 << 8 | 0,&gain, instance_id); //15bit~8bit 用于告知采集板选择通道 1 来读取增益。

读取通道 2 的探测器卡的低能增益:

tymscan_get_gain_high(2 << 8 | 0,&gain, instance_id); //15bit~8bit 用于告知采集板选择通道 2 来读取增益。

[out] gain: 返回该探测器卡高能增益值。

[in] instance: 数据采集板实例 id

[in] bufsize: 设置存储返回的增益值的数组大小。缺省值为 1.

[out] switch_code: 返回目前的 shift 与 test 状态。返回 0, 表示 shift off, test off. 返回 1 表示 shift off, test on. 返回 2 表示 shift on, test off. 返回 3 表示 shift on, test on.

函数: int tymscan_is_support_set_gain_per_card(bool* is_support, int instance)

说明: 判断该产品是否支持给每块探测器卡设置不同的增益

参数:

[out] is_support: 若返回为 true, 则该产品支持给每块探测器卡设置不同的增益。若返回为 false, 则不支持。

[in] instance: 数据采集板实例 id

函数: int tymscan_get_test_pattern_range(int* pattern_min, int* pattern_max, int instance)

说明: 获取测试模式范围

参数:

[out] pattern_min: 测试模式最小值

[out] pattern_max: 测试模式最大值

[in] instance: 数据采集板实例 id

函数: int tymscan_set_test_pattern(int pattern, int instance)

说明: 设置测试模式

参数:

[in] pattern: 测试模式

[in] instance: 数据采集板实例 id

函数: int tymscan_get_test_pattern(int* pattern, int instance)

说明: 获取测试模式

参数:

[out] pattern: 测试模式

[in] instance: 数据采集板实例 id

函数: `int tymscan_get_integral_time_range(int* time_min, int* time_max, int instance)`

说明: 获取积分时间范围

参数:

[out] time_min: 最小积分时间

[out] time_max: 最大积分时间

[in] instance: 数据采集板实例 id

函数: `int tymscan_set_integral_time(int time_us, int instance, int switch_code=0)`

说明: 设置积分时间

参数:

[in] time_us: 积分时间

[in] instance: 数据采集板实例 id

[in] switch_code: 切换码

切换码说明:

对于其他类型系统, switch_code 参数缺省值即可。对于 PANGUC 系统, switch_code 为 0, 表示此时设置的是 trigger time。switch_code 为 1, 表示此时设置的是 integration time。switch_code 为 2, 表示此时设置的是 delay time。

函数: `int tymscan_get_integral_time(int* time_us, int instance, int switch_code = 0)`

说明: 获取积分时间

参数:

[out] time_us: 积分时间

[in] instance: 数据采集板实例 id

[in] switch_code: 切换码

切换码说明:

对于其他类型系统, switch_code 参数缺省值即可。对于 PANGUC 系统, switch_code 为 0, 表示此时获取的是 trigger time。switch_code 为 1, 表示此时获取的是 integration time。switch_code 为 2, 表示此时获取的是 delay time。

函数: `int tymscan_set_daq_ip(const char* ipaddress, int instance, int switch_code = 0)`

说明: 设置数据采集板 ip

参数:

[in] ipaddress: ip 地址

[in] instance: 数据采集板实例 id

[in] switch_code: 切换码

切换码说明:

switch_code 为 0, 表示此时设置的是数据采集板 ip。switch_code 为 1, 表示此时设置的是数据采集板网关。switch_code 为 2, 表示此时设置的是数据采集板子网掩码。

函数: int tymscan_get_daq_ip(char* ipaddress, int ipaddress_length, int instance, int switch_code = 0)

说明: 获取数据采集板 ip

参数:

[out] ipaddress: ip 地址

[in] ipaddress_length: 存储 ip 地址的存储空间长度

[in] instance: 数据采集板实例 id

[in] switch_code: 切换码

切换码说明:

switch_code 为 0, 表示此时获取的是数据采集板 ip。switch_code 为 1, 表示此时获取的是数据采集板网关。switch_code 为 2, 表示此时获取的是数据采集板子网掩码。

函数: int tymscan_set_daq_port(int cmd_port, int img_port, int instance)

说明: 设置数据采集板的指令端口号和图像端口号

参数:

[in] cmd_port: 指令端口号

[in] img_port: 图像端口号

[in] instance: 数据采集板实例 id

函数: int tymscan_get_daq_port(int* cmd_port, int* img_port, int instance)

说明: 获取数据采集板的指令端口号和图像端口号

参数:

[out] cmd_port: 获取到的指令端口号

[out] img_port: 获取到的图像端口号

[in] instance: 数据采集板实例 id

函数: int tymscan_set_pixelnum_percard(int pixelnum, int instance)

说明: 设置每块探测器卡的像素数

参数:

[in] pixelnum: 像素数

[in] instance: 数据采集板实例 id

函数: `int tymscan_get_pixelnum_percard(int* pixelnum, int instance)`

说明: 获取每块探测器卡的像素数

参数:

[out] pixelnum: 像素数

[in] instance: 数据采集板实例 id

函数: `int tymscan_get_pixelnum (int* pixelnum, int instance)`

说明: 获取所有探测器卡总像素数

参数:

[out] pixelnum: 像素数

[in] instance: 数据采集板实例 id

函数: `int tymscan_set_trigger_mode(int mode ,int instance, trigger_info* trig_info=NULLptr)`

说明: 设置触发模式

参数:

[in] mode: 触发模式

mode 可设置 0, 1, 3, 4

0: 表示设置触发模式为内部模式

1: 表示设置触发模式为外部模式

3: 表示设置触发模式为外部模式(倍频输入)

4: 表示设置触发模式为帧触发模式

[in] instance: 数据采集板实例 id

[in] trig_info: 该结构体可用于设置触发信息

结构体定义:

```
typedef struct trigger_info
{
    int pulse_level=1;//表示触发脉冲电平 默认:高电平
    int input_pulse_num = 1;//表示输入脉冲个数, 默认:1
    int output_pulse_width = 50;//表示输出脉冲宽度, 默认:50ms
    int outputLinenum_per_trigger = 512;//表示每触发一次trigger输出多少行数据, 默认:512
}trigger_info_t;
```

函数: `int tymscan_get_trigger_mode(int* mode ,int instance, trigger_info* trig_info=NULLptr)`

说明: 获取触发模式

参数:

[out] mode: 触发模式

mode 的返回值范围为 0, 1, 3, 4

0: 表示此时触发模式为内部模式

1: 表示此时触发模式为外部模式

3: 表示设置触发模式为外部模式(倍频输入)

4: 表示此时触发模式为帧触发模式

[in] instance: 数据采集板实例 id

[out] trig_info: 该结构体可用于读取触发信息

结构体定义:

```
typedef struct trigger_info
{
    int pulse_level=1;//表示触发脉冲电平 默认:高电平
    int input_pulse_num = 1;//表示输入脉冲个数, 默认:1
    int output_pulse_width = 50;//表示输出脉冲宽度, 默认:50ms
    int outputLinenum_per_trigger = 512;//表示每触发一次trigger输出多少行数据, 默认:512
}trigger_info_t;
```

函数: int tymscan_set_energy_mode(int mode ,int instance)

说明: 设置能量模式

参数:

[in] mode: 能量模式

mode 可设置 1, 2, 3, 4

1: 设置单能模式 (PANGU_C 系统不支持该模式)

2: 设置双能模式

3: 设置仅高能模式 (仅 PANGU_C 系统和 PANGU_S 系统支持该模式)

4: 设置仅低能模式 (仅 PANGU_C 系统和 PANGU_S 系统支持该模式)

[in] instance: 数据采集板实例 id

函数: int tymscan_get_energy_mode(int* mode ,int instance)

说明: 获取能量模式

参数:

[out] mode: 能量模式

mode 的返回值范围为 1, 2, 3, 4

1: 表示此时为单能模式 (PANGU_C 系统不支持该模式)

2: 表示此时为双能模式

3: 表示此时为高能模式 (仅 PANGU_C 系统和 PANGU_S 系统支持该模式)

4: 表示此时为低能模式（仅 PANGU_C 系统和 PANGU_S 系统支持该模式）

[in] instance: 数据采集板实例 id

函数: int tymscan_set_cardnumber(int linkid, int number, int instance)

说明: 设置某个槽位的探测器卡数

参数:

[in] linkid: 槽位号

[in] number: 探测器卡数量

[in] instance: 数据采集板实例 id

函数: int tymscan_get_cardnumber(int linkid, int* number, int instance)

说明: 获取某个槽位的探测器卡数

参数:

[in] linkid: 槽位号

[out] number: 探测器卡数量

[in] instance: 数据采集板实例 id

函数: int tymscan_set_datacallback(tymfn_datacallback callback, void* user_data, int block_line_num, int instance, void* data_format = nullptr)

说明: 设置数据回调函数。**注意: 1.请在建立连接成功后调用此函数。2.该函数请在开始采集之前调用, 在采集过程中, 请不要调用此函数。**

参数:

[in] callback: 回调函数指针

[in] user_data: 传入用户上层应用所需的环境指针, 回调函数触发时会将其原封不动传出。

[in] block_line_num: 数据帧块高度。帧块是构成帧的基本单位。用户可根据回调函数所传递的帧块, 来组成自己的图像帧。

[in] instance: 数据采集板实例 id

[in] data_format: 该参数用于配置数据帧格式。通常情况设置缺省值即可。针对有特殊需求的用户, 使用方法请参照后文“特殊说明条目 1”以及“结构体说明条目 4”。

typedef void (__stdcall* tymfn_datacallback)(const tymdata_buffer* buffer, void* extra_info, void* user_data, int instance);

说明: 回调函数指针定义

参数:

buffer: 由回调函数传递过来的图像数据帧结构体

extra_info: 由回调函数传递过来的用户所需的实时状态信息结构体

user_data: 用户上层应用所需的环境指针

instance: 由回调函数传递过来的数据采集板实例 id

函数: `int tymscan_special_set(int cmd_id, int value, int bytes_this_cmd, int instance)`

说明: 针对意料之外的非通用指令, 进行指令数据写入的接口。支持单字节指令和双字节指令

参数:

[in] cmd_id: 指令码

[in] value: 指令数据

[in] bytes_this_cmd: 该指令数据所占字节数

[in] instance: 数据采集板实例 id

函数: `int tymscan_special_get(int cmd_id, int* value, int bytes_this_cmd, int instance)`

说明: 针对意料之外的非通用指令, 进行指令数据读取的接口。支持单字节指令和双字节指令

参数:

[in] cmd_id: 指令码

[out] value: 指令数据

[in] bytes_this_cmd: 该指令数据所占字节数

[in] instance: 数据采集板实例 id

函数: `int tymcan_doOffsetCalibration(int instance)`

说明: 执行 offset 校准

参数:

[in] instance: 数据采集板实例 id

函数: `int tymcan_doGainCalibration(unsigned int target, int instance)`

说明: 执行 gain 校准

参数:

[in] target: 校准目标值

[in] instance: 数据采集板实例 id

函数: `int tymcan_setBaseLine(unsigned int baseLine, int instance)`

说明: 设置 baseline

参数:

[in] baseLine: 基准值

[in] instance: 数据采集板实例 id

函数：int tymcan_set_sendCalibratedData_Enable(bool offset_enable, bool coe_enable, bool baseline_enable, int instance, int* buf_mask_pixs=NULLPTR, int buf_size=0)

说明：使能是否传递校准数据

参数：

[in] offset_enable: 使能 offset 校准

[in] coe_enable: 使能 gain 校准

[in] baseline_enable: 使能 baseline 校准

[in] instance: 数据采集板实例 id

[in] buf_mask_pixs: 用于指定掩蔽像素的数组。注：像素标识以索引形式，从 0 开始。（即 0 代表第 1 个像素）。

[in] buf_size: 掩蔽像素数组的数组大小。

关于掩蔽像素功能的说明：

该功能用于选择哪些像素不执行校准计算，被掩蔽的像素将输出未校准的数据值。

例子：

```
int buf_mask_pixs[] = { 0, 63, -1, 74, -1, 256, 511, -1, 92, -1, 80, -1, 120, 130, -1};
```

```
int buf_size = 15;
```

```
tymcan_set_sendCalibratedData_Enable(true, true, true, instanceId, buf_mask_pixs, buf_size);
```

解释：掩蔽像素数组可以掩蔽多个区间范围的像素。每个区间需要以-1 来结束，表示这个区间已经定义完成。例如：0, 63, -1 表示区间[0, 63]，表示将第 1~64 像素掩蔽。74, -1 表示单独掩蔽第 75 像素。256, 511, -1 表示区间[256, 511]，表示将第 257~512 像素掩蔽。92, -1 表示单独掩蔽第 93 像素。80, -1 表示单独掩蔽第 81 像素。120, 130, -1 表示区间[120, 130]，表示将第 121~131 像素掩蔽。

函数：int tymscan_set_pixel_order_mode(int mode, int instance)

说明：设置像素顺序模式。

参数：

[in] mode: 像素顺序模式

设置为 0: 正常像素顺序

设置为 1: 像素逆序

设置为 2: 通道 1 像素逆序，通道 2 正常像素顺序（仅对 PANGU-SE 平台有效）

设置为 3: 通道 1 正常像素顺序，通道 2 像素逆序（仅对 PANGU-SE 平台有效）

[in] instance: 数据采集板实例 id

函数：int tymscan_save_config (int instance)

说明：保存探测器当前配置，保存后探测器每次上电后将以此配置作为默认值。

用户关心的可保存参数主要包含如下：

1	LED 状态
2	积分时间
3	测试模式
4	系统类型
5	能量模式
6	触发模式
7	每通道连接的探测器数量
8	每块探测器的像素数
9	增益

参数：

[in] instance: 数据采集板实例 id

函数：int tymscan_set_infopage(int channel, int page, const char* filePath, int instance)

说明：设置 PANGU-SE 系统信息页。该功能属于用户定制功能。

参数：

[in] channel: 通道号，可设置范围 1~2

[in] page: 页号，可设置范围 1~4

[in] filePath: 输入的信息页配置文件路径

[in] instance: 数据采集板实例 id

函数：int tymscan_get_infopage(int channel, int page, const char* filePath, int instance)

说明：读取 PANGU-SE 系统信息页。该功能属于用户定制功能。

参数：

[in] channel: 通道号，可设置范围 1~2

[in] page: 页号，可设置范围 1~4

[out] filePath: 输出的信息页配置文件路径

[in] instance: 数据采集板实例 id

函数：int tymscan_set_TH_enable_XH(bool enable, int instance)

说明：使能 PANGU-SE 系统通过回调函数发送温湿度信息。该功能属于用户定制功能。

参数：

[in] enable: 使能。

[in] instance: 数据采集板实例 id

函数：int tymscan_set_custom_pixnum(int pixelnum, int instance)

说明：设置自定义总像素数。该功能属于用户定制功能。

当与校准使能配合使用时，请先执行 `tymcan_set_sendCalibratedData_Enable`，然后执行该函数。

参数：

- [in] pixelnum: 像素数。
- [in] instance: 数据采集板实例 id。

函数：int tymscan_set_detector_type(int detector_type, int instance)

说明：设置探测器类型，此为 PANGU-GS 系统所需函数

参数：

- [in] detector_type: 探测器类型码。定义于以下枚举值：

```
enum DetectorType {  
    PCM08_1DOUT = 1,  
    PCM04_1DOUT = 2,  
    PCM02_1DOUT = 3,  
    PCM01_1DOUT = 4,  
    PCM08DH_2DOUT = 8,  
    PCM04_2DOUT = 9,  
    PCM04DH_2DOUT = 10,  
    PCM02_2DOUT = 11,  
    PCM02DH_2DOUT = 12,  
    PCM01_2DOUT = 13,  
};
```

- [in] instance: 数据采集板实例 id。

函数：int tymscan_get_detector_type(int* detector_type, int instance)

说明：获取探测器类型，此为 PANGU-GS 系统所需函数

参数：

- [out] detector_type: 探测器类型码。定义于以下枚举值：

```
enum DetectorType {  
    PCM08_1DOUT = 1,  
    PCM04_1DOUT = 2,  
    PCM02_1DOUT = 3,  
    PCM01_1DOUT = 4,  
    PCM08DH_2DOUT = 8,  
    PCM04_2DOUT = 9,  
    PCM04DH_2DOUT = 10,  
    PCM02_2DOUT = 11,  
    PCM02DH_2DOUT = 12,  
    PCM01_2DOUT = 13,  
};
```

[in] instance: 数据采集板实例 id。

函数: `int tymscan_enable_auto_get_cardnum(int enable, int instance)`

说明: 使能自动获取板卡数功能。若设置为自动获取板卡数功能, 则 `tymscan_get_cardnumber` 将返回实际连接的板卡个数。若设置为手动获取板卡数功能, 则 `tymscan_get_cardnumber` 将返回 Flash 存储区中存储的板卡个数。

参数:

[int] enable: enable 为 1, 此时设置为自动模式。enable 为 0, 此时设置为手动模式。

[in] instance: 数据采集板实例id。

函数: `int tymscan_is_auto_get_cardnum(int* is_auto, int instance)`

说明: 判断当前的板卡数获取模式。

参数:

[out] is_auto: is_auto 为 1, 表明此时板卡数获取模式为自动模式。is_auto 为 0, 表明此时板卡数获取模式为手动模式。

[in] instance: 数据采集板实例id。

函数: `int tymscan_get_total_lost_lines(unsigned int* total_lost_lines, int instance)`

说明: 获取当前丢失的总行数。

参数:

[out] total_lost_lines: 当前丢失的总行数。

[in] instance: 数据采集板实例id。

函数: `int tymscan_is_attribute_exist(int attribute_id, bool* is_exist, int instance)`

说明: 判断当前使用的产品类型是否具备该属性信息。详细使用方法请参照 9.实时状态信息获取。

参数:

[in] attribute_id: 属性 id, 定义于下列枚举值中:

enum ATTRIBUTE_ID

{

attr_id_lineNumber, //行号

attr_id_data_control_board_temperature, //数据采集板温度

attr_id_data_control_board_humidity, //数据采集板湿度

attr_id_xray_flag, //用于标明该行射线是低能还是高能的标志位。若其值为0, 则表明该

行是高能数据，若其值为1，则表明该行是低能数据。

attr_id_data_mode, //数据模式。0：常数积分模式（不带实时去本底）。

1：像素本底采集模式下的pixeldata。2：像素本底采集模式下的offsetdata

attr_id_voltage1_dcm, //数据采集板的第1个电源信息

attr_id_voltage2_dcm, //数据采集板的第2个电源信息

attr_id_voltage3_dcm, //数据采集板的第3个电源信息

attr_id_voltage4_dcm, //数据采集板的第4个电源信息

attr_id_trigger_pass_reg, //用于判断line trigger和frame trigger是否正常的标志位

attr_id_voltage1_all_fcms, //所有探测器卡的第1个电压信息

attr_id_voltage2_all_fcms, //所有探测器卡的第2个电压信息

attr_id_voltage3_all_fcms, //所有探测器卡的第3个电压信息

attr_id_voltage4_all_fcms, //所有探测器卡的第4个电压信息

attr_id_voltage5_all_fcms, //所有探测器卡的第5个电压信息

attr_id_voltage6_all_fcms, //所有探测器卡的第6个电压信息

attr_id_voltage7_all_fcm, //所有探测器卡的第7个电压信息

attr_id_temperature_all_fcms, //所有探测器卡的温度信息

attr_id_humidity_all_fcms, //所有探测器卡的湿度信息

attr_id_timeStamp, //时间戳信息

attr_id_voltage1_dcm_se, // DCM_SE数据采集板的第1个电源信息

attr_id_voltage2_dcm_se, // DCM_SE数据采集板的第2个电源信息

attr_id_voltage3_dcm_se, // DCM_SE数据采集板的第3个电源信息

attr_id_voltage4_dcm_se, // DCM_SE数据采集板的第4个电源信息

attr_id_voltage5_dcm_se, // DCM_SE数据采集板的第5个电源信息

attr_id_voltage6_dcm_se, // DCM_SE数据采集板的第6个电源信息

attr_id_voltage7_dcm_se, // DCM_SE数据采集板的第7个电源信息

attr_id_voltage8_dcm_se, // DCM_SE数据采集板的第8个电源信息

};

[out] is_exist: 返回 true，则为该属性存在；返回 false，则为该属性不存在。

[in] instance: 数据采集板实例id。

函数：int tymscan_get_attribute_value(int attribute_id, TYM_Variant* var, int instance)

说明：获取某属性的值。详细使用方法请参照 9.实时状态信息获取。

参数：

[in] attribute_id: 属性 id，定义于下列枚举值中：

enum ATTRIBUTE_ID

{

attr_id_lineNumber, //行号

attr_id_data_control_board_temperature, //数据采集板温度

attr_id_data_control_board_humidity, //数据采集板湿度

attr_id_xray_flag, //用于标明该行射线是低能还是高能的标志位。若其值为0，则表明该行是高能数据，若其值为1，则表明该行是低能数据。

```
attr_id_data_mode, //数据模式。0: 常数积分模式（不带实时去本底）。
1: 像素本底采集模式下的pixeldata。2: 像素本底采集模式下的offsetdata
attr_id_voltage1_dcm, //数据采集板的第1个电源信息
attr_id_voltage2_dcm, //数据采集板的第2个电源信息
attr_id_voltage3_dcm, //数据采集板的第3个电源信息
attr_id_voltage4_dcm, //数据采集板的第4个电源信息
attr_id_trigger_pass_reg, //用于判断line trigger和frame trigger是否正常的标志位
attr_id_voltage1_all_fcms, //所有探测器卡的第1个电压信息
attr_id_voltage2_all_fcms, //所有探测器卡的第2个电压信息
attr_id_voltage3_all_fcms, //所有探测器卡的第3个电压信息
attr_id_voltage4_all_fcms, //所有探测器卡的第4个电压信息
attr_id_voltage5_all_fcms, //所有探测器卡的第5个电压信息
attr_id_voltage6_all_fcms, //所有探测器卡的第6个电压信息
attr_id_voltage7_all_fcms, //所有探测器卡的第7个电压信息
attr_id_temperature_all_fcms, //所有探测器卡的温度信息
attr_id_humidity_all_fcms, //所有探测器卡的湿度信息
attr_id_timeStamp, //时间戳信息
attr_id_voltage1_dcm_se, // DCM_SE数据采集板的第1个电源信息
attr_id_voltage2_dcm_se, // DCM_SE数据采集板的第2个电源信息
attr_id_voltage3_dcm_se, // DCM_SE数据采集板的第3个电源信息
attr_id_voltage4_dcm_se, // DCM_SE数据采集板的第4个电源信息
attr_id_voltage5_dcm_se, // DCM_SE数据采集板的第5个电源信息
attr_id_voltage6_dcm_se, // DCM_SE数据采集板的第6个电源信息
attr_id_voltage7_dcm_se, // DCM_SE数据采集板的第7个电源信息
attr_id_voltage8_dcm_se, // DCM_SE数据采集板的第8个电源信息

};
[out] var: 当前属性的值。TYM_Variant 定义如下:
enum TYM_VAR_TYPE //数据类型定义
{
    TYM_INT = 0, //表明属性的返回值是int类型
    TYM_DOUBLE = 1, //表明属性的返回值是double类型
    TYM_USHORT = 2, //表明属性的返回值是unsigned short类型
    TYM_UINT_ARR = 3, //表明属性的返回值是unsigned int*类型
    TYM_UCHAR_ARR = 4, //表明属性的返回值是unsigned char*类型
    TYM_DOUBLE_ARR = 5, //表明属性的返回值是double*类型
    TYM_UCHAR = 6 //表明属性的返回值是unsigned char类型
};
typedef struct TYM_Variant //是一个返回属性的数据类型和取值的数据结构
{
    TYM_VAR_TYPE vt; //当前的数据类型
    int arr_length; //若当前的数据类型为数组, 标明当前数组大小
    struct {
```

```
int val_int;  
double val_double;  
unsigned short val_ushort;  
unsigned int* val_uintArr;  
unsigned char* val_ucharArr;  
double* val_doubleArr;  
unsigned char val_uchar;  
  
}val;  
}TYM_Variant_t;  
[in] instance: 数据采集板实例id。
```

函数: int tymscan_set_work_mode(int mode, int instance, work_mode_param* mode_param = nullptr)

说明: 设置系统工作模式。

参数:

[in] mode: 工作模式。目前共 10 种工作模式。

Mode0: 常规内触发, 根据预设积分时间 (采集频率) 进行连续数据采集。

Mode1: 常规外触发, 根据客户提供的外部触发的信号 (采集频率), 进行连续数据采集。

Mode0(D 系列): 常规内触发, 根据预设积分时间 (采集频率) 进行连续数据采集 (信号积分), 仅限于 PANGU(D)系统产品。

Mode3: 常规外触发, 根据客户提供的外部触发的信号 (采集频率), 进行连续数据采集, 仅限于 PANGU(D)系统产品

*注: 在此模式下外触发信号频率需要设置为实际扫描频率的两倍

Mode4: 帧触发, 根据客户提供的外部触发脉冲及软件所设置的帧高 (图像行数), 以及固定积分的时间, 进行数据发送的模式。

Mode5: 常数积分工作模式, 根据客户外部触发信号开始进行数据采集, 采集周期 (积分时间) 为预设值。应用于传送带速度时变的系统。

Mode6: 编码器分频+常数积分+行数反馈功能模式, 根据客户提供的外部编码器触发脉冲个数进行分频, 产生内触发脉冲并进行常数积分, 并根据图像数据行数, 产生脉冲反馈, 同时提供积分区间屏蔽外部干扰。应用于传送带速度时变的系统, 外部触发产生于客户编码器, 并精确计量位置的系统。

Mode7: 加速器用工作模式, 适用于触发脉冲 (line trigger) 周期较长, 数据采集时间 (积分时间为常数) 很短的工作模式, frame trigger 用于高低能识别, 带实时本底校正, 一次触发, 积分两次

Mode8: 加速器用工作模式, 适用于触发脉冲 (line trigger) 周期较长, 数据采集时间 (积分时间为常数) 很短的工作模式, frame trigger 用于高低能识别, 一次触发, 积分一次

Mode9: 加速器用工作模式, 适用于触发脉冲 (line trigger) 周期较长, 数据采集时间 (积分时间为常数) 很短的工作模式, frame trigger 用于高低能识别, 一次触发, 积分两次, 数据分别上传

Mode10: 外触发戳工作模式, 适用于外触发采用客户提供的脉冲 (line trigger) 字符串

(4bytes+开始停止位)进行触发,同时系统可以将字符串打包在每帧图像的包头进行发送。
frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns

Mode11: 加速器&外触发戳工作模式,适用于外触发采用客户提供的脉冲(line trigger)字符串(4bytes+开始停止位)进行触发,同时系统可以将字符串打包在每帧图像的包头进行发送。frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns,触发脉冲(line trigger)周期较长,数据采集时间(积分时间为常数)很短的工作模式,带实时本底校正,一次触发,积分两次

Mode12: 加速器&外触发戳工作模式,适用于外触发采用客户提供的脉冲(line trigger)字符串(4bytes+开始停止位)进行触发,同时系统可以将字符串打包在每帧图像的包头进行发送。frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns,触发脉冲(line trigger)周期较长,数据采集时间(积分时间为常数)很短的工作模式,带实时本底校正,一次触发,积分一次

Mode13: 加速器用工作模式,适用于本底校准情况,配合 mode8 使用,积分时间时间固定的工作模式,一次触发,积分一次

Mode14: 帧触发+外触发模式,根据客户提供的外部触发脉冲及软件所设置的帧高(图像行数),根据 line_Trigger 进行外触发积分,进行数据发送的模式。

Mode15: 常规外触发,根据客户提供的外部触发的信号(采集频率),进行连续数据采集,仅限于 PANGU(D)系统产品,需要设置抖动时间。

*注: 在此模式下外触发信号频率需要设置为实际扫描频率的两倍。

[in] instance: 数据采集板实例 id。

[in] work_mode_param: 该工作模式下可设的参数。该结构体详细信息请参照: 5 结构体说明

例: 客户预将系统设置为工作模式 0.示例代码如下

```
work_mode_param wk_mode_param; //创建工作模式参数结构体
wk_mode_param.param_mode0.integration_time_us = 1000; //填充模式0的参数
tymscan_set_work_mode(0, instanceId_PANGU, &wk_mode_param); //设置工作模式和参数
```

函数: int tymscan_get_work_mode(int* mode, int instance, work_mode_param* mode_param = nullptr)

说明: 获取系统工作模式。

参数:

[out] mode: 工作模式。目前共 10 种工作模式。

Mode0: 常规内触发,根据预设积分时间(采集频率)进行连续数据采集。

Mode1: 常规外触发,根据客户提供的外部触发的信号(采集频率),进行连续数据采集。

Mode0(D 系列): 常规内触发,根据预设积分时间(采集频率)进行连续数据采集(信号积分),仅限于 PANGU(D)系统产品。

Mode3: 常规外触发,根据客户提供的外部触发的信号(采集频率),进行连续数据采集,仅限于 PANGU(D)系统产品

*注: 在此模式下外触发信号频率需要设置为实际扫描频率的两倍

Mode4: 帧触发,根据客户提供的外部触发脉冲及软件所设置的帧高(图像行数),以及

固定积分的时间，进行数据发送的模式。

Mode5: 常数积分工作模式，根据客户外部触发信号开始进行数据采集，采集周期（积分时间）为预设值。应用于传送带速度时变的系统。

Mode6: 编码器分频+常数积分+行数反馈功能模式，根据客户提供的外部编码器触发脉冲个数进行分频，产生内触发脉冲并进行常数积分，并根据图像数据行数，产生脉冲反馈，同时提供积分区间屏蔽外部干扰。应用于传送带速度时变的系统，外部触发产生于客户编码器，并精确计量位置的系统。

Mode7: 加速器用工作模式，适用于触发脉冲（line trigger）周期较长，数据采集时间（积分时间为常数）很短的工作模式，frame trigger 用于高低能识别，带实时本底校正，一次触发，积分两次

Mode8: 加速器用工作模式，适用于触发脉冲（line trigger）周期较长，数据采集时间（积分时间为常数）很短的工作模式，frame trigger 用于高低能识别，一次触发，积分一次

Mode9: 加速器用工作模式，适用于触发脉冲（line trigger）周期较长，数据采集时间（积分时间为常数）很短的工作模式，frame trigger 用于高低能识别，一次触发，积分两次，数据分别上传

Mode10: 外触发戳工作模式，适用于外触发采用客户提供的脉冲（line trigger）字符串（4bytes+开始停止位）进行触发，同时系统可以将字符串打包在每帧图像的包头进行发送。frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns

Mode11: 加速器&外触发戳工作模式，适用于外触发采用客户提供的脉冲（line trigger）字符串（4bytes+开始停止位）进行触发，同时系统可以将字符串打包在每帧图像的包头进行发送。frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns，触发脉冲（line trigger）周期较长，数据采集时间（积分时间为常数）很短的工作模式，带实时本底校正，一次触发，积分两次

Mode12: 加速器&外触发戳工作模式，适用于外触发采用客户提供的脉冲（line trigger）字符串（4bytes+开始停止位）进行触发，同时系统可以将字符串打包在每帧图像的包头进行发送。frame trigger 用于外部同步时钟,外部同步时钟高电平和低电平时间不低于 100ns，触发脉冲（line trigger）周期较长，数据采集时间（积分时间为常数）很短的工作模式，带实时本底校正，一次触发，积分一次

Mode13: 加速器用工作模式，适用于本底校准情况，配合 mode8 使用，积分时间时间固定的工作模式,一次触发，积分一次

Mode14: 帧触发+外触发模式，根据客户提供的外部触发脉冲及软件所设置的帧高（图像行数），根据 line_Trigger 进行外触发积分，进行数据发送的模式。

Mode15: 常规外触发，根据客户提供的外部触发的信号（采集频率），进行连续数据采集，仅限于 PANGU(D)系统产品，需要设置抖动时间。

*注：在此模式下外触发信号频率需要设置为实际扫描频率的两倍。

[in] instance: 数据采集板实例 id。

[out] work_mode_param: 该工作模式下获取的当前参数信息。该结构体详细信息请参照：
5 结构体说明

例：客户预读取当前的系统工作模式及当前参数。

```
int mode_get = -1;
```

```
work_mode_param wk_mode_param_get;
```

```
tymscan_get_work_mode(&mode_get, instanceId_PANGU,&wk_mode_param_get);
```

函数：int tymscan_get_hardware_info_file(const char* output_filepath, int instance, int auth_code=0)

说明：获取硬件信息，并把所有信息输出到 txt 文件中。

参数：

[out] output_filepath: 输出文件的路径。

[in] instance: 数据采集板实例 id。

[in] auth_code: 权限码。用户使用缺省值即可。

函数：int tymscan_get_hardware_info(int hwinfo_id, char* info_buf, int info_buf_length, int instance, int auth_code = 0)

说明：获取硬件信息。

参数：

[in] hwinfo_id: 硬件信息 id，用于选择期望获取的硬件信息项目。

id 定义：

enum HARDWARE_INFO_ID

{

fcminfo_id_smt_pcba_sns, //此id用于获取所有探测器卡(FCM)的SMT生产的PCBA序列号

fcminfo_id_smt_tpn, //此id用于获取所有探测器卡(FCM)的SMT生产的TYM部件号

fcminfo_id_smt_hardware_versions, //此id用于获取所有探测器卡(FCM)的SMT生产的硬件版本号

fcminfo_id_smt_manufact_dates, //此id用于获取所有探测器卡(FCM)的SMT生产的生产日期

fcminfo_id_smt_supplier_names, //此id用于获取所有探测器卡(FCM)的SMT生产的供应商缩写

fcminfo_id_tym_detector_module_sns, //此id用于获取所有探测器卡(FCM)的TYM生产的探测器模块序列号

fcminfo_id_tym_hardware_versions, //此id用于获取所有探测器卡(FCM)的TYM生产的硬件版本号

fcminfo_id_tym_manufact_dates, //此id用于获取所有探测器卡(FCM)的TYM生产的生产日期

fcminfo_id_tym_supplier_names, //此id用于获取所有探测器卡(FCM)的TYM生产的供应商缩写

fcminfo_id_tym_detector_module_class, //此id用于获取所有探测器卡(FCM)的TYM生产的探测器模块等级

dcminfo_id_smt_pcba_sn, //此id用于获取数据采集板(DCM)的SMT生产的PCBA序列号

dcminfo_id_smt_tpn, //此id用于获取数据采集板(DCM)的SMT生产的TYM部件号

dcminfo_id_smt_hardware_version, //此id用于获取数据采集板(DCM)的SMT生产的硬件

版本号

```
    dcminfo_id_smt_manufact_date, //此id用于获取数据采集板(DCM)的SMT生产的生产日期
    dcminfo_id_smt_supplier_name, //此id用于获取数据采集板(DCM)的SMT生产的供应商缩写
};
```

[out] info_buf: 用于获取硬件信息的数组。

[in] info_buf_length: 用于获取硬件信息的数组大小。

[in] instance: 数据采集板实例 id。

[in] auth_code: 权限码。用户使用缺省值即可。

示例：用户期望获取所有探测器卡的探测器模块序列号

```
    int length_fcm_tym_sn = 0;
    tymscan_get_hardware_info_length(fcminfo_id_tym_detector_module_sns,
    &length_fcm_tym_sn, instanceId_PANGU); //获取该项目的字符串长度
    char* fcm_tym_sn = new char[length_fcm_tym_sn + 1]; //根据长度创建数组，用于存储序列号
    tymscan_get_hardware_info(fcminfo_id_tym_detector_module_sns, fcm_tym_sn,
    length_fcm_tym_sn, instanceId_PANGU); //获取序列号
    std::cout << fcm_tym_sn << std::endl;
    delete[] fcm_tym_sn;
    fcm_tym_sn = nullptr;
```

函数：int tymscan_get_hardware_info_length(int hwinfo_id, int* info_length, int instance)

说明：获取指定项的硬件信息的字符串长度。与 tymscan_get_hardware_info 函数配合使用。

参数：

[in] hwinfo_id: 硬件信息 id，用于选择期望获取的硬件信息项目。

id 定义：

enum HARDWARE_INFO_ID

{

fcminfo_id_smt_pcba_sns, //此id用于获取所有探测器卡(FCM)的SMT生产的PCBA序列号

fcminfo_id_smt_tpsns, //此id用于获取所有探测器卡(FCM)的SMT生产的TYM部件号

fcminfo_id_smt_hardware_versions, //此id用于获取所有探测器卡(FCM)的SMT生产的硬件版本号

fcminfo_id_smt_manufact_dates, //此id用于获取所有探测器卡(FCM)的SMT生产的生产日期

fcminfo_id_smt_supplier_names, //此id用于获取所有探测器卡(FCM)的SMT生产的供应商缩写

fcminfo_id_tym_detector_module_sns, //此id用于获取所有探测器卡(FCM)的TYM生产的探测器模块序列号

fcminfo_id_tym_hardware_versions, //此id用于获取所有探测器卡(FCM)的TYM生产的硬件版本号

fcminfo_id_tym_manufact_dates, //此id用于获取所有探测器卡(FCM)的TYM生产的生产日期

fcminfo_id_tym_supplier_names, //此id用于获取所有探测器卡(FCM)的TYM生产的供应商缩写

fcminfo_id_tym_detector_module_class, //此id用于获取所有探测器卡(FCM)的TYM生产的探测器模块等级

dcminfo_id_smt_pcba_sn, //此id用于获取数据采集板(DCM)的SMT生产的PCBA序列号

dcminfo_id_smt_tpn, //此id用于获取数据采集板(DCM)的SMT生产的TYM部件号

dcminfo_id_smt_hardware_version, //此id用于获取数据采集板(DCM)的SMT生产的硬件版本号

dcminfo_id_smt_manufact_date, //此id用于获取数据采集板(DCM)的SMT生产的生产日期

dcminfo_id_smt_supplier_name, //此id用于获取数据采集板(DCM)的SMT生产的供应商缩写
};

[out] info_length: 指定项的硬件信息的字符串长度。

[in] instance: 数据采集板实例 id。

函数: int tymscan_get_fcm_firmware_version(unsigned short* version, int fcm_qty, int instance)

说明: 获取所有探测器模块的固件版本号

参数:

[out] version: 返回所有探测器模块的固件版本号

[in] fcm_qty: 探测器模块的数量

[in] instance: 数据采集板实例 id

函数: int tymscan_set_status_callback(tymfn_status_callback callback, void* user_data, int time_us, int instance)

说明: 设置状态信息回调函数。请在建立连接成功后调用此函数。详细使用方法, 请参照 9 实时状态信息获取。

参数:

[in] callback: 回调函数指针

[in] user_data: 传入用户上层应用所需的环境指针, 回调函数触发时会将其原封不动传出。

[in] time_us: 两次回调函数的间隔时间。用户可通过此参数来控制每隔多长时间, 触发一次回调函数。

[in] instance: 数据采集板实例 id

typedef void (__stdcall* tymfn_status_callback)(void* user_data, int instance);

说明: 回调函数指针定义。详细使用方法, 请参照 9 实时状态信息获取。

参数:

user_data: 用户上层应用所需的环境指针

instance: 由回调函数传递过来的数据采集板实例 id

函数: `int tymscan_pdc_set_pitch_and_gaps(double pitch, double* gaps, int buf_size, int instance)`

说明: 将当前产品的 pitch 大小与卡间像素间距写入配置文件中。若配置文件已生成, 则无需再次调用该函数。

注意: 为保证该函数成功调用, 在调试程序时, 请将校准目录 **calib_dir** 放到您所编程序的源码目录下, 程序发布时, 请将该目录放到即将发布的程序的软件根目录下

参数:

[in] pitch: pitch 大小。

[in] gaps: 所有卡的卡间像素间距。例如: 12 块卡, 则有 11 个卡间像素间距。

[in] buf_size: 用于存储 gaps 的数组大小。

[in] instance: 数据采集板实例 id

例子: 客户连接了 12 块卡, 有 11 个卡间像素间距。

```
double gaps[] = { 0.286,0.218,0.353,0.272,0.327,0.332,0.223,0.315,0.243,0.222,0.191 };
double pitch = 0.1;
tymscan_pdc_set_pitch_and_gaps(pitch, gaps, 11, instanceId);
```

函数: `int tymscan_pdc_enable(bool enable, int instance)`

说明: 使能是否进行 PDC 校正。若使能, 则传递的图像数据将为进行 PDC 校正后的数据。

注意: 为保证该函数成功调用, 在调试程序时, 请将校准目录 **calib_dir** 放到您所编程序的源码目录下, 程序发布时, 请将该目录放到即将发布的程序的软件根目录下

参数:

[in] enable: 使能 PDC 校正

[in] instance: 数据采集板实例 id

函数: `int tymscan_verify_fcm_info(int instance)`

说明: 该函数用于判断 FCM 之间保存的关键参数是否一致。当 FCM 之间关键参数不一致时, 将通过返回值来指明不一致的参数。当 FCM 之间的关键参数一致, 但 FCM 与 DCM 之间的该参数不一致时, 则将会把 FCM 的该参数信息, 覆写到 DCM 中。

返回值含义:

0XF01: 存在不一致的 FCM 系统类型

0XF02: 存在不一致的 FCM 像素数量

0XF03: 存在不一致的 FCM 能量模式

0XF04: 存在不一致的 FCM 操作模式

0XF05: 存在不一致的 FCM 像素顺序

0XF06: 存在不一致的 FCM 指示灯状态

0XF07: 存在不一致的 FCM 探测器类型

0XF08: 存在不一致的 FCM ASIC MODE

参数:

[in] instance: 数据采集板实例 id

函数: int tymscan_set_fcm_info(unsigned short fcm_addr, int slot_id, const char* fcm_info, int instance)

说明: 向 FCM 中设置参数信息。

参数:

[in] fcm_addr: FCM 的地址空间的偏移地址

[in] slot_id: 槽位号, 范围 1~4

[in] fcm_info: 写入的 FCM 的参数信息, 为字符串

[in] instance: 数据采集板实例 id

函数: int tymscan_get_fcm_info(unsigned short fcm_addr, int slot_id, char* fcm_info_buf, int buf_length, int instance)

说明:

参数:

[in] fcm_addr: FCM 的地址空间的偏移地址

[in] slot_id: 槽位号, 范围 1~4

[out] fcm_info_buf: 用于存放从 FCM 读出的参数信息, 返回的字符串将存入该字符数组中

[in] buf_length: 字符数组 fcm_info_buf 的大小

[in] instance: 数据采集板实例 id

函数: int tymscan_enable_interpolation (bool enable, int instance, unsigned int max_allow_lost_lines = 1)

说明: 使能是否进行丢行插值。若使能, 则当丢行数量小于等于 max_allow_lost_lines 行时, 进行丢失行补全。

注意: 该功能只支持 PANGU 平台系列产品

参数:

[in] enable: 使能丢行插值

[in] instance: 数据采集板实例 id

[in] max_allow_lost_lines: 当丢行数量小于等于 max_allow_lost_lines 行时, 进行丢失行补全。缺省值为 1: 表示当丢行数量等于 1 行时, 进行丢失行补全。

函数: `int tymscan_get_detector_type_str(char* detector_type, int detector_type_length, int instance)`

说明: 获取当前所连探测器的探测器类型。

注意: 若此时获取的探测器类型为 `UnKnow Type`, 说明此时 FCM 中存储的探测器类型存在不一致的情况, 用户可调用函数 `tymscan_get_detector_type_all_fcms_length` 和 `tymscan_get_detector_type_all_fcms` 来获取每个 FCM 的探测器类型。(通常情况无需调用 `tymscan_get_detector_type_all_fcms_length` 函数和 `tymscan_get_detector_type_all_fcms` 函数)

参数:

[out] `detector_type`: 用于获取探测器类型的数组。

[in] `detector_type_length`: 用于获取探测器类型的数组大小。

[in] `instance`: 数据采集板实例 id。

函数: `int tymscan_get_detector_type_all_fcms_length(int* info_length, int instance)`

说明: 获取所有 FCM 的探测器类型信息的字符串长度。与 `tymscan_get_detector_type_all_fcms` 函数配合使用。

参数:

[out] `info_length`: 所有 FCM 的探测器类型信息的字符串长度。

[in] `instance`: 数据采集板实例 id。

函数: `int tymscan_get_detector_type_all_fcms(char* detector_type_all_fcms, int detector_type_all_fcms_length, int instance)`

说明: 获取所有 FCM 的探测器类型信息。

参数:

[out] `detector_type_all_fcms`: 用于获取所有 FCM 的探测器类型信息的数组。

[in] `detector_type_all_fcms_length`: 用于获取所有 FCM 的探测器类型信息的数组大小。

[in] `instance`: 数据采集板实例 id。

示例: 用户期望获取所有 FCM 的探测器类型信息。

```
int length_detector_all_fcms = 0;
```

```
tymscan_get_detector_type_all_fcms_length(&length_detector_all_fcms, instanceId_PANGU);
```

```
//获取所有FCM的探测器类型信息的字符串长度
```

```
char* detector_all_fcms = new char[length_detector_all_fcms+1]; //根据长度创建数组, 用于  
存储所有FCM的探测器类型信息
```

```
tymscan_get_detector_type_all_fcms(detector_all_fcms, length_detector_all_fcms,  
instanceId_PANGU); //获取所有FCM的探测器类型信息
```

```
std::cout << detector_all_fcms << std::endl;
```

```
delete[] detector_all_fcms;
```

```
detector_all_fcms = nullptr;
```

函数：int tymscan_get_status_info_all(tymscan_status_info_all* stinfo_all, int instance)

说明：获取状态信息。

参数：

[in] stinfo_all: 状态信息结构体，用于获取当前产品所有的状态信息。（若当前产品不支持获取某状态，则结构体中与该状态相关的数值型变量为 0，数组指针为空指针）

结构体定义：

```
typedef struct tymscan_status_info_all
```

```
{
```

```
    double status_data_control_board_temperature;//数据采集板的温度信息
```

```
    double status_data_control_board_humidity;//数据采集板的湿度信息
```

```
    double* status_data_control_board_voltages;//数据采集板的电压信息，由数组存储
```

```
    int num_of_vols_data_control_board;//存储数据采集板电压信息的数组大小
```

```
    int num_of_detectors;//探测器卡数量
```

```
    double* status_temperature_all_detectors;//每个探测器卡的温度信息，由数组存储，数组大小为探测器数量
```

```
    double* status_humidity_all_detectors;//每个探测器卡的湿度信息，由数组存储，数组大小为探测器数量
```

```
    int num_of_vols_per_detector;//每个探测器所包含的电压信息数量。
```

```
    double** status_voltages_all_detectors;//每个探测器卡的电压信息。第一层指针指向某块探测器卡的电压信息数组，第二层指针指向电压信息数组的某个电压。
```

```
}tymscan_status_info_all_t;
```

[in] instance: 数据采集板实例 id。

示例：用户期望获取当前产品的所有状态信息

```
tymscan_status_info_all stinfo_all;
```

```
tymscan_get_status_info_all(&stinfo_all, instanceId_PANGU);
```

```
std::cout << "data_control_board_temperature:" <<
```

```
stinfo_all.status_data_control_board_temperature<<std::endl;
```

```
std::cout << "data_control_board_humidity:" <<
```

```
stinfo_all.status_data_control_board_humidity << std::endl;
```

```
for (int i = 0; i < stinfo_all.num_of_detectors; i++)
```

```
{
```

```
    std::cout << "FCM"+std::to_string(i+1)+" temperature:" <<
```

```
stinfo_all.status_temperature_all_detectors[i] << std::endl;
```

```
}

    for (int i = 0; i < stinfo_all.num_of_detectors; i++)
    {
        std::cout << "FCM" + std::to_string(i + 1) + " humidity:" <<
stinfo_all.status_humidity_all_detectors[i] << std::endl;
    }

    for (int i = 0; i < stinfo_all.num_of_vols_data_control_board; i++)
    {
        std::cout << "data_control_board voltage" + std::to_string(i + 1) + ":" <<
stinfo_all.status_data_control_board_voltages[i] << std::endl;
    }

    for (int i = 0; i < stinfo_all.num_of_detectors; i++)
    {
        for (int j = 0; j < stinfo_all.num_of_vols_per_detector; j++)
        {
            std::cout << "FCM" + std::to_string(i + 1) + " voltage" +
std::to_string(j + 1) + ":" << stinfo_all.status_voltages_all_detectors[i][j] <<
std::endl;
        }
    }
}
```

函数：int tymscan_set_calib_timeout_for_extrig(int time_out_ms, int instance)

说明：此功能用于外触发下校准设置超时时间。（请在执行校准之前调用该函数）

参数：

[in] time_out_ms: 超时时间，单位毫秒

[in] instance: 数据采集板实例 id。

5 结构体说明

1, typedef struct tymdata_buffer

```
{
    int line_num;
    int pixels_per_line;
    int bytes_per_pixel;
```

```
int bytes_per_line;  
unsigned char* data;  
} tymimage_buffer_t;
```

说明：由回调函数传递过来的图像数据帧结构体。

参数：

line_num: 该帧数据的行数
pixels_per_line: 每行数据的像素数
bytes_per_pixel: 每个像素所占的字节数
bytes_per_line: 每行的字节数
data: 图像数据

2, typedef struct userdata_daq

```
{  
    int height;  
  
    unsigned int* lineNumber;
```

```
}userdata_daq_t;
```

说明：由回调函数传递过来的用户所需的实时状态信息结构体。此结构体适用于 DAQ,DAQII,DAQIIEX,PANGU_SE,PANGU_S 系统

参数：

height: 该帧数据的行数
lineNumber: 该帧数据的行号

3, typedef struct userdata_pangu

```
{  
    int height;  
    int blocknum;  
    unsigned int* lineNumber;  
    unsigned short* temperature;  
    unsigned short* humidity;  
    unsigned short* voltage1;  
    unsigned short* voltage2;  
    unsigned short* voltage3;  
    unsigned short* voltage4;
```

```
}userdata_pangu_t;
```

说明：由回调函数传递过来的用户所需的实时状态信息结构体。此结构体传递的是 PANGU 系统的信息

参数：

height: 该帧数据的行数
blocknum: 该帧数据包含的帧块数
lineNumber: 该帧数据的行号
temperature: 该帧数据的温度信息
humidity: 该帧数据的湿度信息
voltage1, voltage2, voltage3, voltage4: 该帧数据的电压信息

```
4, typedef struct userdata_panguc
{
    userdata_pangu usrdata_pangu;
    unsigned char* xray_flag;
    unsigned char* data_mode;
```

```
}userdata_panguc_t;
```

说明: 由回调函数传递过来的用户所需的实时状态信息结构体。此结构体传递的是 PANGUC 系统的信息

参数:

usrdata_pangu: 该结构体引用了 PANGU 系统的结构体
xray_flag: 表示该帧数据各行射线照射状态信息
xray_flag 的返回值范围为 0 和 1
0: 表示此时是高能射线
1: 表示此时是低能射线
data_mode: 表示该帧数据各行的数据模式信息
data_mode 的返回值范围为 0, 1, 2
0: 表示此时是常积分模式 (不带实时去本底)
1: 表示此时是像素本底采集模式下的 pixeldata
2: 表示此时是像素本底采集模式下的 offsetdata

```
5, typedef struct info_header_setting {

    int null_byte_num;
    unsigned char* buf_item_order;
    int bufsize_item_order;
    enum item_id { id_linenum, id_null_byte};
```

```
}info_header_setting_t;
```

说明: 该结构体可作为函数 `tymscan_set_datacallback` 的参数 `data_format` 传入。用于向图像数据的每一行添加指定格式的信息头。目前信息头最大支持设定 100 字节。当前可支持填充进信息头的条目包括: 每一行的行号, 若干数目的空字节。使用示例请参照特殊说明条目 1。

参数:

- null_byte_num: 指明想要填充的空字节数目
- buf_item_order: 填入想要的条目顺序, 条目由条目 id 表示。
- bufsize_item_order: buf_item_order 所指向的数组的大小
- enum item_id: 条目 id, 请勿更改。

6, typedef struct work_mode_param

```
{  
    param_for_work_mode0 param_mode0;  
    param_for_work_mode1 param_mode1;  
    param_for_work_mode3 param_mode3;  
    param_for_work_mode4 param_mode4;  
    param_for_work_mode5 param_mode5;  
    param_for_work_mode6 param_mode6;  
    param_for_work_mode7 param_mode7;  
    param_for_work_mode8 param_mode8;  
    param_for_work_mode9 param_mode9;  
    param_for_work_mode10 param_mode10;  
    param_for_work_mode11 param_mode11;  
    param_for_work_mode12 param_mode12;  
    param_for_work_mode13 param_mode13;  
    param_for_work_mode14 param_mode14;  
    param_for_work_mode15 param_mode15;
```

```
}work_mode_param_t;
```

说明: 因每种工作模式的参数均有区别, 故创建此结构体, 用于方便用户快速设置指定模式的参数值。

默认值:

```
const int edge_trigger_default=0x03;//默认为上升沿  
const int discarded_image_lines_per_frame_default = 5;//每帧图像丢弃行数, 默认丢弃5行  
const int image_lines_per_frame_default = 512;//每帧图像行数, 默认为512行  
const int timestamp_collection_edge_default = 0x00;  
const int parity_default = 0x00;
```

结构体:

```
typedef struct param_for_work_mode0  
{  
    int integration_time_us;//积分时间  
  
}param_for_work_mode0_t;
```

```
typedef struct param_for_work_model
{
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int external_trigger_filtering_time_threshold_us; //外触发屏蔽阈值

}param_for_work_model_t;

typedef struct param_for_work_mode3
{
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int external_trigger_filtering_time_threshold_us; //外触发屏蔽阈值

}param_for_work_mode3_t;

typedef struct param_for_work_mode4
{
    int image_lines_per_frame = image_lines_per_frame_default; //每帧图像行数
    int integration_time_us; //积分时间
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int discarded_image_lines_per_frame = discarded_image_lines_per_frame_default; // 每帧
    图像丢弃行数

}param_for_work_mode4_t;

typedef struct param_for_work_mode5
{
    int constant_integration_time_us; //常数积分时间
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int external_trigger_filtering_time_threshold_us; //外触发屏蔽阈值

}param_for_work_mode5_t;

typedef struct param_for_work_mode6
{
    int times_of_frequency_division; //分频倍数
    int constant_integration_time_us; //常数积分时间
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int external_trigger_filtering_time_threshold_us; //外触发屏蔽阈值
    int feedback_pulse_of_image_acquisition_quantity; // 每采集多少行输出反馈脉冲
    int feedback_pulse_width_ms; // 反馈脉冲宽度

}param_for_work_mode6_t;
```

```
typedef struct param_for_work_mode7
{
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int delay_time_from_external_trigger_to_integration_start_us; // 积分开始相对外触发延时
    int interval_between_xray_offset_sampling; //射线本底积分时间间隔
    int corrected_baseline_average_times; //减本底平均次数，取值1/2/4/8
    int integration_time_us; //积分时间
    //注意： interval_between_xray_offset_sampling与integration_time_us之和，需大于
    600us
}param_for_work_mode7_t;
```

```
typedef struct param_for_work_mode8
{
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int delay_time_from_external_trigger_to_integration_start_us; // 积分开始相对外触发延时
    int integration_time_us; //积分时间

}param_for_work_mode8_t;
```

```
typedef struct param_for_work_mode9
{
    int edge_trigger = edge_trigger_default; //帧/外触发沿： 0x03为上升沿， 0x00为下降沿
    int delay_time_from_external_trigger_to_integration_start_us; // 积分开始相对外触发延时
    int interval_between_xray_offset_sampling; //射线本底积分时间间隔
    int integration_time_us; //积分时间
    //注意： interval_between_xray_offset_sampling与integration_time_us之和，需大于
    600us
}param_for_work_mode9_t;
```

```
typedef struct param_for_work_mode10
{
    int timestamp_collection_edge = timestamp_collection_edge_default; //时间戳采集沿： 0x03
    为上升沿， 0x00 为下降沿
    int parity = parity_default; //奇偶校验设置:0x00 为不校验， 0x02 为偶校验， 0x01 为奇校
    验
}param_for_work_mode10_t;
```

```
typedef struct param_for_work_mode11
{
    int timestamp_collection_edge = timestamp_collection_edge_default; //时间戳采集沿： 0x03
    为上升沿， 0x00为下降沿
```

```
int delay_time_from_external_trigger_to_integration_start_us;//积分开始相对外触发延时
int interval_between_xray_offset_sampling;//射线本底积分时间间隔
int corrected_baseline_average_times;//减本底平均次数，取值1/2/4/8
int integration_time_us;//积分时间
int parity = parity_default;//奇偶校验设置:0x00为不校验，0x02为偶校验，0x01为奇校验

}param_for_work_model1_t;

typedef struct param_for_work_model2
{
    int timestamp_collection_edge = timestamp_collection_edge_default;//时间戳采集沿：0x03
    为上升沿，0x00为下降沿
    int delay_time_from_external_trigger_to_integration_start_us;//积分开始相对外触发延时
    int integration_time_us;//积分时间
    int parity = parity_default;//奇偶校验设置:0x00为不校验，0x02为偶校验，0x01为奇校验

}param_for_work_model2_t;

typedef struct param_for_work_model3
{
    int trigger_cycle_us;//触发时间
    int integration_time_us;//积分时间

}param_for_work_model3_t;

public struct param_for_work_model4
{
    public int image_lines_per_frame;//每帧图像行数
    public int edge_trigger;//帧/外触发沿：0x03为上升沿，0x00为下降沿
    public int discarded_image_lines_per_frame;// 每帧图像丢弃行数
};

typedef struct param_for_work_model5
{
    int edge_trigger = edge_trigger_default;//帧/外触发沿：0x03为上升沿，0x00为下降沿
    int integration_time_us;//外触发周期
    double jitt_rate;//抖动率。例如：若抖动率为千分之1，请设置为0.001

}param_for_work_model5_t;
```

结构体使用方法, 请参照 4 函数说明中的 `tymscan_set_work_mode` 和 `tymscan_get_work_mode`。

7, typedef struct tymscan_status_info_all

```
{
    double status_data_control_board_temperature;//数据采集板的温度信息
    double status_data_control_board_humidity;//数据采集板的湿度信息

    double* status_data_control_board_voltages;//数据采集板的电压信息，由数组存储
    int num_of_vols_data_control_board;//存储数据采集板电压信息的数组大小

    int num_of_detectors;//探测器卡数量
    double* status_temperature_all_detectors;//每个探测器卡的温度信息，由数组存储，数组
    大小为探测器数量
    double* status_humidity_all_detectors;//每个探测器卡的湿度信息，由数组存储，数组大
    小为探测器数量

    int num_of_vols_per_detector;//每个探测器所包含的电压信息数量。
    double** status_voltages_all_detectors;//每个探测器卡的电压信息。第一层指针指向某块
    探测器卡的电压信息数组，第二层指针指向电压信息数组的某个电压。

}tymscan_status_info_all_t;
```

说明：该结构体用于获取当前产品的状态信息。与 `tymscan_get_status_info_all` 函数配合使用。
详情请参照 4.函数说明。

6 返回值说明

TYM_NOERROR = 0: 返回正确

TYM_TIMEOUT = -1: 返回超时

TYM_RETURNERR = -2: 返回错误

TYM_PRODUCT_NO_SUCH_FUNCTION = 0XEE: 表示该产品类型不支持当前功能

TYM_CMDSENDFORBIDDEN = 0XFB: 禁止发送该指令

TYM_CMDDATAOUTOFRANGE = 0XAA: 指令数据超出允许范围

TYM_CMDWRONGDATALENGTH = 0XAE: 指令 `Data_Length` 设置错误

TYM_INSTANCE_INVALID = 0XCC: 数据采集板实例 id 无效

TYM_WRITE_READ_MISMATCH = 0XAB: 当内部读写校验不匹配时，将返回该值

TYM_FCM_SYSTYPE_NOT_SAME = 0XF01: 存在不一致的 FCM 系统类型

TYM_FCM_PIXNUM_PERCARD_NOT_SAME = 0XF02: 存在不一致的 FCM 像素数量

TYM_FCM_ENERGYMODE_NOT_SAME = 0XF03: 存在不一致的 FCM 能量模式

TYM_FCM_OPMODE_NOT_SAME = 0XF04: 存在不一致的 FCM 操作模式

TYM_FCM_PIXREVERSE_NOT_SAME = 0XF05: 存在不一致的 FCM 像素顺序

TYM_FCM_LEDCTRL_NOT_SAME = 0XF06: 存在不一致的 FCM 指示灯状态

TYM_FCM_DETECTOR_TYPE_NOT_SAME = 0XF07: 存在不一致的 FCM 探测器类型
TYM_FCM_ASICMODE_NOT_SAME=0XF08: 存在不一致的 FCM ASIC MODE

TYM_CALIB_GET_CORRECT_DIR_FAILED = 1: 获取校准文件夹失败
TYM_CALIB_CANT_OPEN_OFFSET_FILE = 2: 无法打开offset校准文件
TYM_CALIB_OFFSET_PIXNUM_DIFF = 3: 当前的像素数与offset校准文件中的像素数不一致
TYM_CALIB_CANT_OPEN_COE_FILE = 4: 无法打开coe校准文件
TYM_CALIB_COE_PIXNUM_DIFF = 5: 当前的像素数与coe校准文件中的像素数不一致
TYM_CALIB_CANT_OPEN_BASELINE_FILE = 6: 无法打开 baseline 校准文件
TYM_CALIB_MASK_INDEX_OUT_OF_RANGE = 7:要屏蔽的像素索引超出了最大范围

7 校准流程说明

在执行校准流程之前，请先阅读如下说明

校准目录说明：

校准目录为 `calib_dir`。当用户使用校准方式 2 时，为保证校准相关函数成功调用，在调试程序时，请将校准目录放到您所编程序的源码目录下，程序发布时，请将该目录放到即将发布的程序的软件根目录（可执行文件所在目录）下。当用户使用校准方式 1 时，校准目录会在执行校准函数的过程中自动生成，用户无需关心校准目录问题。

用户可通过两种方式完成校准：

- 1, 用户通过在自己所编程序的源码中调用校准相关函数，执行校准。
- 2, 用户通过由我司提供的昆仑图像采集软件执行校准，然后将该软件根目录下的 `calib_dir` 文件夹，直接复制到用户所编程序的源码目录下和即将发布的程序根目录下。然后在用户源码中调用 `tymcan_set_sendCalibratedData_Enable` 函数（该函数请在与探测器建立连接成功之后调用）。

1, 用户通过调用校准相关函数，执行校准的方式：

（下列函数，请在 `tymscan_set_datacallback` 函数执行后调用，并且请不要在采集过程中，调用下列函数）

- (1) 请确保关闭射线，执行 `offset` 校准。执行 `offset` 校准请调用函数 `tymcan_doOffsetCalibration`。校准完成后，会在校准目录下，生成 `offset` 校准文件。
- (2) 开启射线（请确保射线源与探测器之间无障碍物），执行 `gain` 校准。执行 `gain` 校准请调用函数 `tymcan_doGainCalibration`。校准完成后，会在校准目录下，生成 `coe` 校准文件。
- (3) 用户可根据需要，设置基准值。如需设置基准值，请调用函数 `tymcan_setBaseLine`。设置完成后，会在校准目录下，生成 `baseline` 文件。
- (4) 若想直接获取校准后的数据，请根据需要，使能所需的校准项。请调用函数 `tymcan_set_sendCalibratedData_Enable`

8 特殊说明

1, 结构体 `info_header_setting` 使用示例。

用户想在图像数据的每一行插入一个 8 字节的信息头。该 8 字节信息头前四字节为该行行号, 后 4 字节为空字节。实现代码如下:

```
info_header_setting header_setting;
unsigned char buf_item_order[2];//创建一个用于存储指定顺序条目 id 的数组
buf_item_order[0] = header_setting.id_linenum;//行号作为第一个条目填充到信息头
buf_item_order[1] = header_setting.id_null_byte;//空字节作为第二个条目填充到信息头。
header_setting.buf_item_order = buf_item_order;//结构体指针指向用于存储指定顺序条目
id 的数组
header_setting.bufsize_item_order = 2;// buf_item_order 这个数组的大小为 2
header_setting.null_byte_num = 4;//指明想要填充 4 个空字节
tymscan_set_datacallback(callback, 128, instanceId_PANGU,&header_setting);//作为参数填入函数
```

9 实时状态信息获取

功能说明：实时状态信息获取功能，旨在获取例如温湿度等状态信息。因实时状态信息仅在图像采集过程中才会传递，故实时状态信息需从回调函数中调用相关函数进行获取。

获取实时状态信息，需使用以下两个函数：

函数：int tymscan_is_attribute_exist(int attribute_id, bool* is_exist, int instance)

函数：int tymscan_get_attribute_value(int attribute_id, TYM_Variant* var, int instance)

用例说明 1：该用例旨在获取实时传递的图像帧行号信息。因行号信息的传递频率与图像的传递频率需要保持一致，故请在传递图像的回调函数中，调用上述函数。

```
void __stdcall callback(const tymdata_buffer* buffer, void* extra_info, void* user_data,
int instance)
{
    if (instance == instanceId_PANGU)
    {

        bool attr_is_exist = false;
        tymscan_is_attribute_exist(attr_id_lineNumber, &attr_is_exist,
instanceId_PANGU);
        if (attr_is_exist)
        {
            TYM_Variant tv;
            tymscan_get_attribute_value(attr_id_lineNumber, &tv, instanceId_PANGU);
            unsigned int* value = tv.val.val_uintArr;
            for (int i = 0; i < tv.arr_length; i++)
            {
                std::cout << "line_number:" << value[i] << std::endl;
            }
        }
    }
}
```

用例说明2：该用例旨在获取所有探测器卡的温湿度信息。因温湿度信息传递频率不需要与图像的传递频率强关联，故用户可根据需要设定状态信息回调函数的触发频率。

```
void __stdcall callback(const tymdata_buffer* buffer, void* extra_info, void* user_data,
int instance);//图像数据回调函数声明
```

```
void __stdcall callback_status(void* user_data, int instance);//状态信息回调函数声明
```

```
void show_status(TYM_Variant* tv, string attr_name);//用于打印显示状态信息的函数声明
```

```
tymscan_set_datacallback(callback, nullptr, 32, instanceId_PANGU);//图像数据回调请务必绑定
```

```
tymscan_set_status_callback(callback_status, nullptr, 1000, instanceId_PANGU);//绑定状态信息回调函数，并设置回调函数每1000ms触发一次。
```

tymscan_grab_start(instanceId_PANGU);// 实时状态信息仅在图像采集过程中才会传递，故需要执行采集。

//回调函数定义

```
void __stdcall callback_status(void* user_data, int instance)
```

```
{
```

```
    if (instance == instanceId_PANGU)
```

```
    {
```

```
        bool attr_is_exist_temperature_all_fcms = false;
```

```
        tymscan_is_attribute_exist(attr_id_temperature_all_fcms,
```

```
&attr_is_exist_temperature_all_fcms, instanceId_PANGU);//判断当前产品类型是否支持该属性的传递
```

```
        if (attr_is_exist_temperature_all_fcms)
```

```
        {
```

```
            TYM_Variant tv;
```

```
            tymscan_get_attribute_value(attr_id_temperature_all_fcms, &tv,
```

```
instanceId_PANGU);//获取属性值
```

```
            show_status(&tv, "temperature_of_fcm");//打印显示属性值
```

```
        }
```

```
        bool attr_is_exist_humidity_all_fcms = false;
```

```
        tymscan_is_attribute_exist(attr_id_humidity_all_fcms,
```

```
&attr_is_exist_humidity_all_fcms, instanceId_PANGU); //判断当前产品类型是否支持该属性的传递
```

```
        if (attr_is_exist_humidity_all_fcms)
```

```
{
    TYM_Variant tv;

    tymscan_get_attribute_value(attr_id_humidity_all_fcms, &tv,
instanceId_PANGU); //获取属性值

    show_status(&tv, "humidity_of_fcm");//打印显示属性值
}
}
```

//用于打印显示状态信息的函数定义

```
void show_status(TYM_Variant* tv, string attr_name)
{
    switch (tv->vt)
    {
    case TYM_INT:
    {
        int value = tv->val.val_int;
        std::cout << attr_name << ":" << value << std::endl;
    }
    break;
    case TYM_DOUBLE:
    {
        double value = tv->val.val_double;
        std::cout << attr_name << ":" << value << std::endl;
    }
    break;
    case TYM_USHORT:
    {
        unsigned short value = tv->val.val_ushort;
        std::cout << attr_name << ":" << value << std::endl;
    }
    break;
    case TYM_UINT_ARR:
    {
        unsigned int* value = tv->val.val_uintArr;

        for (int i = 0; i < tv->arr_length; i++)
        {
            std::cout << attr_name << ":" << value[i] << std::endl;
        }
    }
}
```

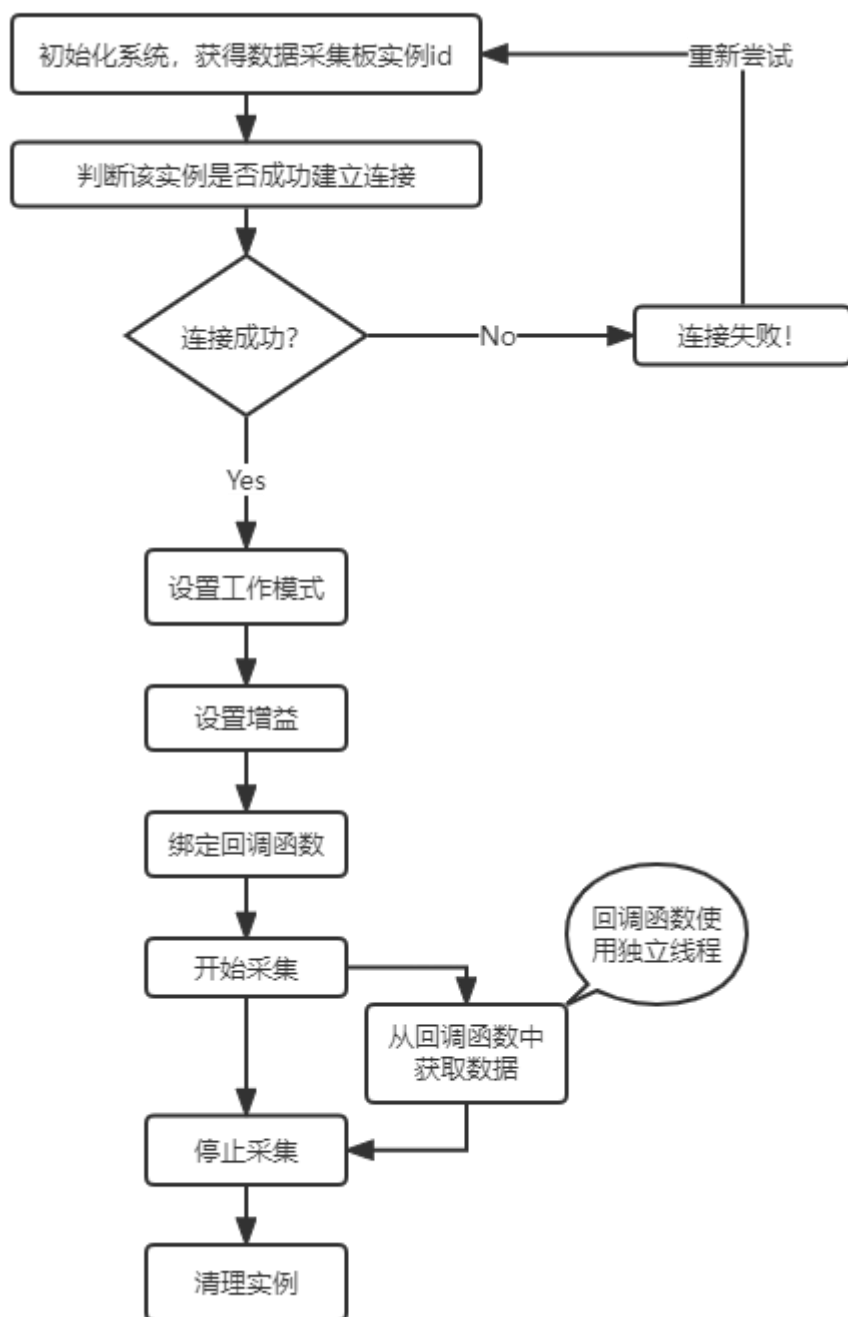
```
break;
case TYM_UCHAR_ARR:
{
    unsigned char* value = tv->val.val_ucharArr;
    for (int i = 0; i < tv->arr_length; i++)
    {
        std::cout << attr_name << (i + 1) << ":" << (int)value[i] << std::endl;
    }
}
break;

case TYM_DOUBLE_ARR:
{
    double* value = tv->val.val_doubleArr;
    for (int i = 0; i < tv->arr_length; i++)
    {
        std::cout << attr_name << (i + 1) << ":" << value[i] << std::endl;
    }
}
break;

case TYM_UCHAR:
{
    unsigned char value = tv->val.val_uchar;
    std::cout << attr_name<<":"<<(int)value << std::endl;
}
break;

default:
    break;
}
}
```

10 系统快速使用流程图



11 示例代码

以下示例代码演示了简单的 Win32 控制台程序使用动态链接库的方式，便于用户快速了解动态库的基本用法：

```
#include <iostream>
#include "tydatabuffer.h"
#include "tymscandaq.h"
#include <thread>

using namespace std;

void __stdcall callback(const tydata_buffer* buffer, void* extra_info, void* user_data,
int instance);//回调函数声明

int instanceId_PANGU = -1;//数据采集板实例id

int main(int argc, char* argv[])
{
    tymscan_init("192.168.10.100", "192.168.10.1", 7171, 7474, &instanceId_PANGU);//初始化数据采集板实例，内部包括与数据采集板建立连接

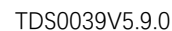
    bool connected_PANGU = false;

    tymscan_connected(&connected_PANGU, instanceId_PANGU);//检测数据采集板是否连接成功

    if (connected_PANGU)
    {
        //注：若该产品为单能级产品，只需设置tymscan_set_gain_low，而无需设置
        tymscan_set_gain_high
        //注：增益可配置范围可由函数tymscan_get_gain_range读出
        tymscan_set_gain_low(0, 6, instanceId_PANGU);//设置低能增益
        tymscan_set_gain_high(0, 6, instanceId_PANGU);//设置高能增益

        tymscan_set_integral_time(3000, instanceId_PANGU);//设置积分时间

        tymscan_set_datacallback(callback, nullptr, 32, instanceId_PANGU);//设置回调函数，表示每接收到32行数据，触发一次回调函数。
```



```
tymscan_grab_start(instanceId_PANGU); //启动采集

std::this_thread::sleep_for(std::chrono::minutes(1));

tymscan_grab_stop(instanceId_PANGU); //停止采集

tymscan_cleanup(instanceId_PANGU); //清理实例，内部包括与数据采集板断开连接，释放内存等操作
}

system("pause");

}

void handle(const tymdata_buffer* buf) //该函数演示打印图像buffer中的每个像素值。
{
    int height = buf->line_num;
    int width = buf->pixels_per_line;
    for (int i = 0; i < height; i++)
    {
        unsigned char* lineAddress = buf->data + i * buf->bytes_per_line; //读取该图像buffer的行首地址

        for (int j = 0; j < width; j++)
        {
            unsigned int val = (((unsigned short*)lineAddress) + j); //获取该行图像的每个像素的像素值
            printf("%d\t", val);

        }
        printf("\n");
    }
    printf("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX\nXXXXXXXXXXXXXXXXXXXX");
}

void __stdcall callback(const tymdata_buffer* buffer, void* extra_info, void* user_data, int instance) //回调函数
{
    if (instance == instanceId_PANGU)
```

```
{

    handle(buffer);

}

}
```

12 数据格式

用户从回调函数中获取到的数据格式如下：

- (1) 假设用户连接了 3 块单能探测器卡，每块探测器卡 64 像素，回调函数设置每次传递 32 行数据，此时数据格式如下所示（每个绿色长条表示一行数据）：

	第一块探测器卡数据 (pixel:1~64)	第二块探测器卡数据 (pixel:65~128)	第三块探测器卡数据 (pixel:129~192)
line1			
line2			
line3			
...			
...			
line32			

- (2) 假设用户连接了 3 块双能探测器卡，每块探测器卡 128 像素（低能 64 像素+高能 64 像素），回调函数设置每次传递 32 行数据，此时数据格式如下所示（其中，每行由所有探测器卡的低能（绿色）和所有探测器卡的高能（蓝色）组成）：

	第一块探测器卡低能数据 (pixel:1~64)	第二块探测器卡低能数据 (pixel:65~128)	第三块探测器卡低能数据 (pixel:129~192)	第一块探测器卡高能数据 (pixel:193~256)	第二块探测器卡高能数据 (pixel:257~320)	第三块探测器卡高能数据 (pixel:321~384)
line1						
line2						
line3						
...						
...						
line32						